Aplicações de Processamento Digital de Sinais – 4456S-04

Prática 5: Projeto de Filtros IIR

Antônio Vinicius Corrêa dos Santos

21/09/2026

Sinal: `all_star.wav`, $F_a = 44\,100$ Hz, $F_a/2 = 22\,050$ Hz. Requisitos: $A_{\text{máx}} = 0{,}5$ dB, $A_{\text{mín}} = 40$ dB.

1 Espectro de frequências de $x(n)$

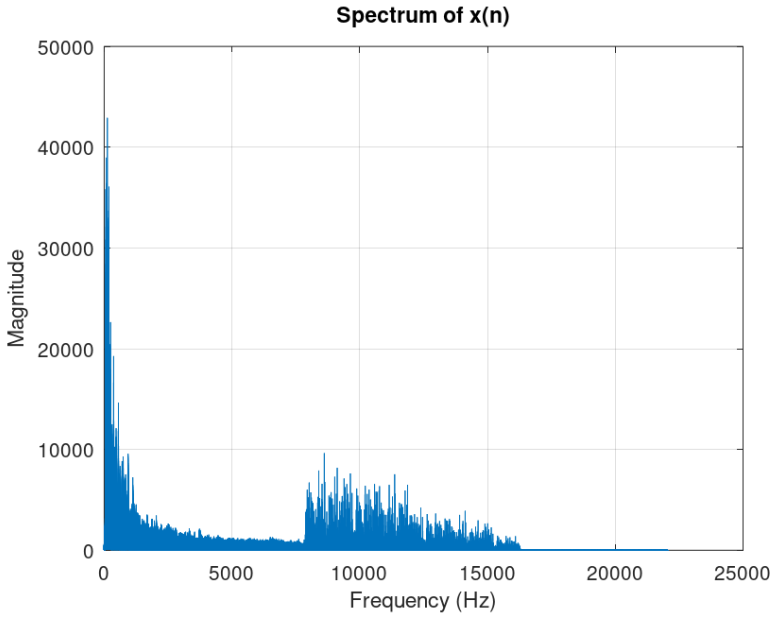


Figura 1: Espectro de magnitude de $x(n)$.

Divisão adotada para os filtros, a mesma da Prática 4:

```
0               600               7800           12500
|---------------|-----------------|--------------|-----------------> f(Hz)
   passa-baixas     rejeita-banda     passa-banda     passa-altas
```

2 Projeto dos filtros IIR

Aproximações: Butterworth no passa-baixas e no rejeita-banda, que são os filtros que deixam passar a região de baixa frequência, onde está a maior parte da energia de $x(n)$ e onde o *ripple* seria audível; Chebyshev I no passa-altas e no passa-banda, onde a transição é mais apertada e a economia de ordem compensa a ondulação de 0,5 dB.

Método: pré-distorção das bordas, protótipo normalizado (`cheb1ap`), transformação de banda (`lp2lp`, `lp2hp`, `lp2bp`, `lp2bs`) e transformação bilinear.

As bandas de transição são mais largas que as da Prática 4, porque o IIR é bem mais seletivo que o FIR por janelamento e uma transição estreita só infla a ordem. No passa-banda e no rejeita-banda a quarta borda vem da simetria geométrica exigida pela transformação, $\Omega_{s2} = \Omega_{p1}\Omega_{p2}/\Omega_{s1}$.

Tabela 1: Especificações e resultados de projeto.

Filtro	**Aprox.**	**Passagem [Hz]**	**Rejeição [Hz]**	n	**Ordem**	$\lvert H\rvert$ **nas bordas [dB]**
Passa-baixas	Butt.	600	1200	9	9	$-0{,}50$ / $-45{,}19$
Passa-altas	Cheb. I	12500	10000	8	8	$-0{,}50$ / $-47{,}23$
Passa-banda	Cheb. I	7800–12500	6500 e 13952,9	6	12	$-0{,}50$ / $-42{,}91$
Rejeita-banda	Butt.	< 300 e $> 12536{,}5$	600–7800	8	16	$-0{,}50$ / $-42{,}81$

3 Respostas em frequência sobrepostas

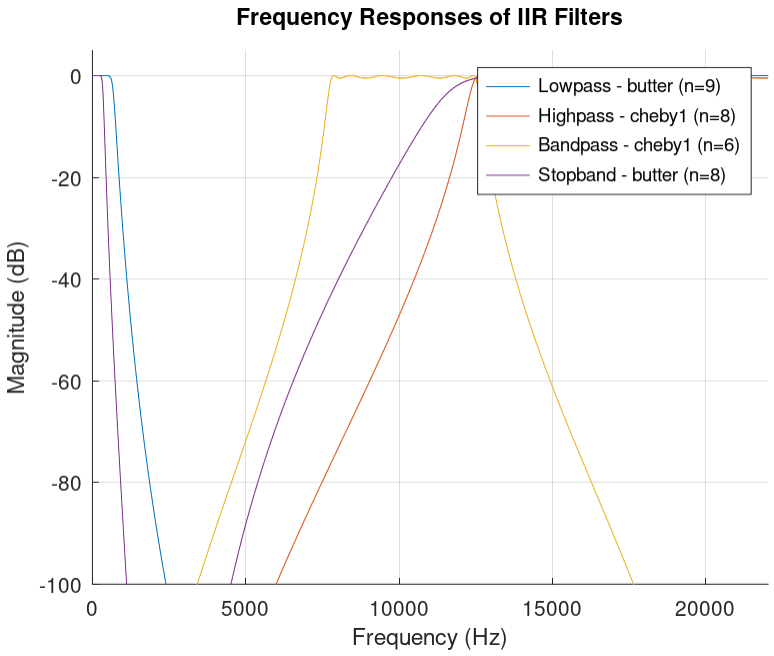


Figura 2: Respostas em frequência dos quatro filtros IIR.

4 Espectros de saída e validação

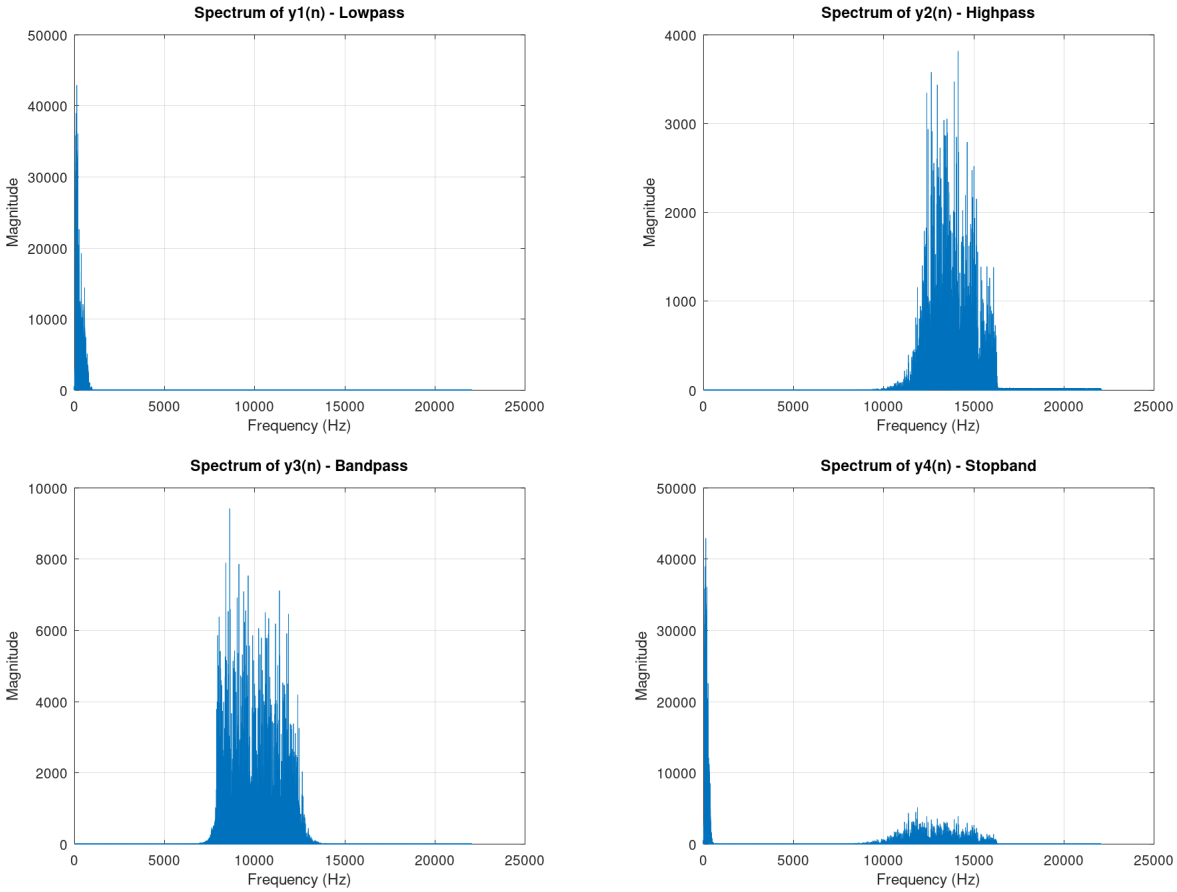


Figura 3: Espectros de y_1 (passa-baixas), y_2 (passa-altas), y_3 (passa-banda) e y_4 (rejeita-banda).

Tabela 2: Banda prevista pelas especificações × banda observada, tomada onde $|Y|$ fica a menos de 40 dB do seu pico.

Saída	**Banda prevista [Hz]**	**Banda observada [Hz]**
y_1	0–600	0–936
y_2	> 12500	10218–16333
y_3	7800–12500	7330–13340
y_4	< 300 e > 12536	0–16225

Tomando uma frequência dentro da banda passante e outra dentro da banda de rejeição de cada filtro, e comparando a magnitude do espectro antes e depois da filtragem:

Tabela 3: Magnitude do espectro em duas frequências por filtro, antes e depois da filtragem.

Filtro	**Banda**	f **[Hz]**	$\lvert X(f)\rvert$	$\lvert Y(f)\rvert$	**Atenuação [dB]**
Passa-baixas	passante	300	$1{,}43 \times 10^{3}$	$1{,}43 \times 10^{3}$	0,0
	rejeição	3000	713	$9{,}15 \times 10^{-4}$	117,8
Passa-altas	passante	15000	148	141	0,5
	rejeição	3000	713	$1{,}54 \times 10^{-5}$	153,3
Passa-banda	passante	10000	286	271	0,5
	rejeição	3000	713	$2{,}72 \times 10^{-3}$	108,4
Rejeita-banda	passante	150	$9{,}16 \times 10^{3}$	$9{,}16 \times 10^{3}$	0,0
	rejeição	3000	713	$2{,}68 \times 10^{-6}$	168,5

Nas frequências da banda passante a magnitude atravessa o filtro praticamente intacta, com perda de no máximo 0,5 dB, dentro de $A_{\text{máx}} = 0{,}5\,\text{dB}$. Nas frequências da banda de rejeição ela é eliminada, com atenuação de 108 a 168 dB, muito além dos $A_{\text{mín}} = 40\,\text{dB}$ exigidos — $A_{\text{mín}}$ é cobrado na borda da banda, e essas frequências estão no interior dela. Os quatro filtros funcionam como especificado.

Validação:

- As bordas observadas caem dentro das bandas de transição especificadas: y_1 termina em 936 Hz (transição 600–1200 Hz), y_2 começa em 10 218 Hz (10 000–12 500 Hz) e y_3 ocupa 7330–13 340 Hz (transições 6500–7800 e 12 500–13 952,9 Hz).
- O limite superior de y_2 e y_4, em torno de 16,3 kHz, é imposto pelo conteúdo de $x(n)$, que já é nulo acima dessa frequência, e não pelos filtros.
- O pico de y_1 e de y_4 é $4{,}29 \times 10^{4}$, idêntico ao pico de $x(n)$, confirmando ganho unitário (0 dB) na banda passante.

5 Atraso de grupo e polos e zeros

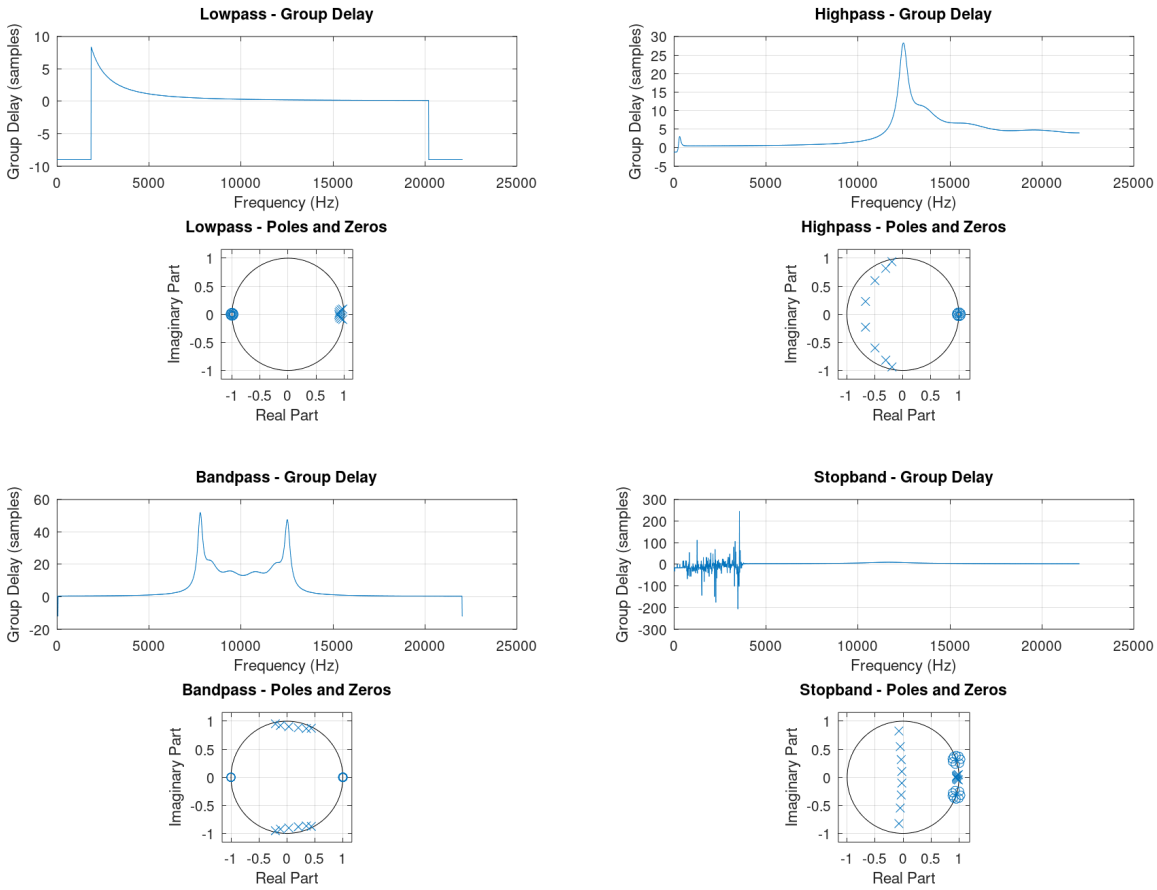


Figura 4: Atraso de grupo e diagrama de polos e zeros: passa-baixas, passa-altas, passa-banda e rejeita-banda.

Ao contrário dos FIR da Prática 4, o atraso de grupo não é constante: é aproximadamente plano no interior da banda passante e sobe junto à borda de transição, onde os polos ficam mais próximos da circunferência unitária. Os máximos são 8,4 amostras em 1852 Hz no passa-baixas, 28 em 12 468 Hz no passa-altas, 52 em 7784 Hz no passa-banda e 246 em 3553 Hz no rejeita-banda. Este último é o maior de todos, mas cai dentro da banda rejeitada, já atenuada em mais de 40 dB, e portanto não distorce a saída. A fase não linear é o preço do IIR: as ordens aqui são uma fração das dos FIR equivalentes da Prática 4.

Todos os polos estão dentro da circunferência unitária (maior módulo 0,983, 0,958, 0,977 e 0,991), logo os quatro filtros são estáveis. Como os dois protótipos são só de polos, todos os zeros de $H(z)$ caem sobre a circunferência unitária: em $z = -1$ ($f = F_a/2$) no passa-baixas, em $z = +1$ ($f = 0$) no passa-altas, metade em cada um no passa-banda, e na frequência central da banda rejeitada no rejeita-banda. Nos gráficos eles aparecem como um aglomerado em vez de um ponto único porque um zero de multiplicidade alta é numericamente sensível.

6 Sistema completo e teste dos ganhos

Implementado $y(n) = \sum_{k=1}^{4} G_k y_k(n)$ conforme a Figura 1 do enunciado da Prática 4. Comutando cada G_k entre "0" e "1":

- G_1 (passa-baixas): som abafado, só o grave; o vocal ainda é inteligível.

- G_2 (passa-altas): amplitude muito baixa (rms 0,0257), só o "chiado" da música.
- G_3 (passa-banda): sinal fraco e metálico (rms 0,0341), acompanha o ritmo sem vocal nem instrumentos definidos.
- G_4 (rejeita-banda): o vocal quase desaparece, sobram o grave e o agudo.
- $G_1 = G_2 = G_3 = G_4 = 1$: parecido com $x(n)$, mas com o meio da banda enfraquecido; a faixa 1200–6500 Hz não é coberta por nenhum filtro com ganho pleno e as somas nas transições ocorrem com fases diferentes.

Script (Octave)

```
%{
    Lab 5 - IIR Filter Design and Analysis

    name: antonio
    date: 21/09/2026
%}

close all
clear all
clc

pkg load signal control

Amax = 0.5; % dB, max attenuation in the passband
Amin = 40;  % dB, min attenuation in the stopband

% 'off' keeps the figure windows from opening; they are saved to img/ either
% way. It has to be passed on every figure() call, which ignores the default.
VIS = 'off';

%% Plot Spectrum of a Signal
function [S, f] = plot_spectrum(sig, Fs, fig_id, ttl, vis)
    m = length(sig);
    n = pow2(nextpow2(m));
    S = abs(fft(sig, n));
    f = (0:n-1)*(Fs/n);

    S = S(1:n/2);
    f = f(1:n/2);

    figure(fig_id, 'visible', vis);
    plot(f, S);
    xlabel('Frequency (Hz)');
    ylabel('Magnitude');
    title(ttl);
    grid on;
end

%% Normalized Lowpass Prototype
function [b, a, n, W] = prototype(approx, Wp, Ws, Amax, Amin)
    % b, a : prototype H(s)
    % n    : prototype order
    % W    : frequency the prototype has to be scaled to by the band
    %        transformation

    switch approx
        % maximally flat passband, higher order
        case 'butter'
            [n, W] = buttord(Wp, Ws, Amax, Amin, 's'); % W: 3 dB frequency
            [z, p, k] = buttap(n);

        % ripple of Amax in the passband, lower order
        case 'cheby1'
            n = cheb1ord(Wp, Ws, Amax, Amin, 's');
            [z, p, k] = cheb1ap(n, Amax);
            W = Wp; % the prototype cutoff is the edge of the ripple band

```

```
        otherwise
            error('prototype: unknown approximation: %s', approx);
    end

    b = poly(z)*k;
    a = poly(p);
end

%% Filter Design
function [b, a, n, edges] = design_iir(type, approx, specs, Fs, Amax, Amin);
    % IIR filter design: lowpass, highpass, bandpass, stopband
    % approximation: Butterworth or Chebyshev I
    %
    % the band edges are pre-warped as Wa = 2*Fs*tan(pi*f/Fs) so that the
    % bilinear transform brings them back to where they were specified
    %
    % n     : order of the lowpass prototype (H(z) has order 2*n on the
    %         bandpass and stopband filters)
    % edges : [passband stopband] edges in Hz, after symmetry

    switch type
        % lowpass
        case 'low'
            Wp = 2*Fs*tan(pi*specs.fp/Fs);
            Ws = 2*Fs*tan(pi*specs.fs/Fs);

            [bp, ap, n, W] = prototype(approx, Wp, Ws, Amax, Amin);
            [bs, as] = lp2lp(bp, ap, W);

            edges = [specs.fp specs.fs];

        % highpass
        case 'high'
            Wp = 2*Fs*tan(pi*specs.fp/Fs);
            Ws = 2*Fs*tan(pi*specs.fs/Fs);

            [bp, ap, n, W] = prototype(approx, Wp, Ws, Amax, Amin);
            [bs, as] = lp2hp(bp, ap, W);

            edges = [specs.fp specs.fs];

        % bandpass
        case 'bandpass'
            Wp1 = 2*Fs*tan(pi*specs.fp1/Fs); Wp2 = 2*Fs*tan(pi*specs.fp2/Fs);
            Ws1 = 2*Fs*tan(pi*specs.fs1/Fs);
            Ws2 = Wp1*Wp2/Ws1; % geometric symmetry, required by lp2bp

            [bp, ap, n, W] = prototype(approx, [Wp1 Wp2], [Ws1 Ws2], Amax, Amin);

            % center frequency and bandwidth
            W0 = sqrt(W(1)*W(2));
            B  = W(2) - W(1);

            [bs, as] = lp2bp(bp, ap, W0, B);

            fs2 = Fs*atan(Ws2/(2*Fs))/pi;
            edges = [specs.fp1 specs.fp2 specs.fs1 fs2];

        % stopband
        case 'stop'
            Ws1 = 2*Fs*tan(pi*specs.fs1/Fs); Ws2 = 2*Fs*tan(pi*specs.fs2/Fs);
            Wp1 = 2*Fs*tan(pi*specs.fp1/Fs);
            Wp2 = Ws1*Ws2/Wp1; % geometric symmetry, required by lp2bs

            [bp, ap, n, W] = prototype(approx, [Wp1 Wp2], [Ws1 Ws2], Amax, Amin);

            % center frequency and bandwidth
            W0 = sqrt(W(1)*W(2));
            B  = W(2) - W(1);

            [bs, as] = lp2bs(bp, ap, W0, B);

            fp2 = Fs*atan(Wp2/(2*Fs))/pi;
```

```
            edges = [specs.fp1 fp2 specs.fs1 specs.fs2];

        otherwise
            error('design_iir: unknown filter type: %s', type);
    end

    [b, a] = bilinear(bs, as, 1/Fs); % octave's bilinear takes T, not Fs
end

file = 'all_star.wav';

% read samples
[x,Fs] = audioread(file);
x = x(:,1);

%% Spectrum of the original signal
[X, f] = plot_spectrum(x, Fs, 1, 'Spectrum of x(n)', VIS);
saveas(gcf, 'img/spectrum_x.png');

% struct to pack filter specifications
filters = struct('name', {}, 'type', {}, 'approx', {}, 'specs', {}, 'test', {});

%% Filter Specifications
% same band split of lab 4, with wider transition bands: an IIR filter is
% far more selective than a windowed FIR, so a narrow transition only
% inflates the order
% |------------|-----------------|-----------------|------------------> f(Hz)
% 0  lowpass  600   stopband   7800   bandpass   12500   highpass
%
% approximation: Butterworth on the two filters that pass the low band,
% where most of the energy of x(n) is and passband ripple would be audible;
% Chebyshev I on the other two, where it saves the most order

% Lowpass filter: fp=600 Hz, fs=1200 Hz
filters(1).name   = 'Lowpass';
filters(1).type   = 'low';
filters(1).approx = 'butter';
filters(1).specs = struct('fp', 600, 'fs', 1200);
filters(1).test  = [  300 3000]; % one frequency inside each band, to check
                                 % the filtering at the end of the script

% Highpass filter: fp=12500 Hz, fs=10000 Hz
filters(2).name   = 'Highpass';
filters(2).type   = 'high';
filters(2).approx = 'cheby1';
filters(2).specs = struct('fp', 12500, 'fs', 10000);
filters(2).test  = [15000 3000];

% Bandpass filter: fp1=7800 Hz, fp2=12500 Hz, fs1=6500 Hz (fs2 by symmetry)
filters(3).name   = 'Bandpass';
filters(3).type   = 'bandpass';
filters(3).approx = 'cheby1';
filters(3).specs = struct('fp1', 7800, 'fp2', 12500, 'fs1', 6500);
filters(3).test  = [10000 3000];

% Stopband filter: fp1=300 Hz, fs1=600 Hz, fs2=7800 Hz (fp2 by symmetry)
filters(4).name   = 'Stopband';
filters(4).type   = 'stop';
filters(4).approx = 'butter';
filters(4).specs = struct('fp1', 300, 'fs1', 600, 'fs2', 7800);
filters(4).test  = [  150 3000];

%% Filter Design
nfilt = numel(filters);
b     = cell(1, nfilt);
a     = cell(1, nfilt);
n     = zeros(1, nfilt);
edges = cell(1, nfilt);

for k = 1:nfilt
    fprintf('Designing filter %d/%d: %s (%s)\n', k, nfilt, filters(k).name, filters(k).
        approx);
    [b{k}, a{k}, n(k), edges{k}] = ...
```

```
        design_iir(filters(k).type, filters(k).approx, filters(k).specs, Fs, Amax, Amin);

    % |H| at the band edges: >= -Amax in the passband, <= -Amin in the stopband
    H = freqz(b{k}, a{k}, edges{k}, Fs);
    fprintf('  n = %d, H(z) order = %d, |H| at %s Hz = %s dB\n', n(k), numel(a{k})-1, ...
            mat2str(round(edges{k}*10)/10), mat2str(round(20*log10(abs(H(:)'))*100)/100))
                ;
end

%% Frequency Responses
figure(2, 'visible', VIS);
hold on;
legend_names = cell(1, nfilt);
for k = 1:nfilt
    [H, w] = freqz(b{k}, a{k}, 2048, Fs);
    plot(w, 20*log10(abs(H) + eps));
    legend_names{k} = sprintf('%s - %s (n=%d)', filters(k).name, filters(k).approx, n(k))
        ;
end
hold off;
axis([0 Fs/2 -100 5]);
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Frequency Responses of IIR Filters');
legend(legend_names);
grid on;
saveas(gcf, 'img/freq_responses.png');

%% Group Delay
for k = 1:nfilt
    figure(2+k, 'visible', VIS);

    subplot(2,1,1);
    [gd, w] = grpdelay(b{k}, a{k}, 2048, Fs);
    plot(w, gd);
    xlabel('Frequency (Hz)');
    ylabel('Group Delay (samples)');
    title(sprintf('%s - Group Delay', filters(k).name));
    grid on;

    subplot(2,1,2);
    zplane(b{k}, a{k});
    title(sprintf('%s - Poles and Zeros', filters(k).name));
    saveas(gcf, sprintf('img/filter_%d_pz.png', k));

    [gd_max, i] = max(gd);
    fprintf('%s: max group delay = %.1f samples at %.0f Hz, max |pole| = %.3f\n', ...
            filters(k).name, gd_max, w(i), max(abs(roots(a{k}))));
end

%% Filtering and Filter Spectrum
y_k = cell(1, nfilt);

for k = 1:nfilt
    fprintf('Filtering with %s...\n', filters(k).name);
    y_k{k} = filter(b{k}, a{k}, x);
    [Y, f] = plot_spectrum(y_k{k}, Fs, 6+k, ...
                sprintf('Spectrum of y%d(n) - %s', k, filters(k).name), VIS);
    saveas(gcf, sprintf('img/spectrum_y%d.png', k));

    % observed band: where |Y| is less than Amin below its own peak
    i = find(Y > 10^(-Amin/20)*max(Y));
    fprintf('  peak |Y| = %.3g (peak |X| = %.3g), band = %.0f - %.0f Hz, rms = %.4f\n',
        ...
            max(Y), max(X), f(i(1)), f(i(end)), rms(y_k{k}));

    % does the filter do its job? take one frequency inside the passband and
    % one inside the stopband and compare the spectrum before and after
    label = {'passband', 'stopband'};
    for r = 1:2
        [~, j] = min(abs(f - filters(k).test(r)));
        fprintf('  %s %5.0f Hz: |X| = %8.3g, |Y| = %8.3g, attenuation = %6.1f dB\n', ...
                label{r}, filters(k).test(r), X(j), Y(j), -20*log10(Y(j)/X(j)));
```

```
273     end
274 end
275
276 %% Diagram
277 %      lp   hp   bp   sb
278 G = [   1    1    1    1 ]; % gains for each filter (lp, hp, bp, sb)
279 y = zeros(size(x));
280
281 for k = 1:nfilt
282     y = y + G(k) * y_k{k};
283 end
284
285 %% Play Filtered Signals
286
287 player_lp = audioplayer(y_k{1}, Fs); % som abafado, so o grave. Da pra entender o vocal,
        mas os instrumentos somem.
288 player_hp = audioplayer(y_k{2}, Fs); % som muito baixo e agudo, so o "chiado" da musica.
        Nao da pra entender o vocal.
289 player_bp = audioplayer(y_k{3}, Fs); % som baixo e metalico, acompanha o ritmo mas sem
        vocal nem instrumentos definidos.
290 player_sb = audioplayer(y_k{4}, Fs); % vocal quase some, sobra o grave e o agudo. Parece
        um audio "oco".
291
292 player_y  = audioplayer(y, Fs);      % parecido com o original, mas com o meio da banda
        enfraquecido.
293 player_clean = audioplayer(x, Fs);   % audio original, sem filtragem.
```

Listing 1: `main.m`